

EVIDENCIAS DE AJUSTES IMPLEMENTADOS

Sistema de Gestión de Ventas

Django REST Framework · PostgreSQL · 13 ajustes obligatorios

Tecnológico en Desarrollo de Software

Evidencias de Ajustes Implementados

El presente documento muestra el fragmento de código fuente exacto donde se implementa cada uno de los 13 ajustes obligatorios, junto con el espacio designado para insertar la captura de pantalla correspondiente.

1. Swagger — Documentación automática

Archivo	config/settings.py
Librería	drf-spectacular
URL de acceso	http://localhost:8000/api/docs/
Descripción	Genera automáticamente la documentación OpenAPI 3.0 de todos los endpoints. Se configura el título, versión, descripción y las etiquetas por módulo.

Captura de evidencia —

```
# — Swagger / drf-spectacular —
SPECTACULAR_SETTINGS = {
    'TITLE': 'API Sistema de Gestión de Ventas',
    'DESCRIPTION': (
        'API REST para administrar clientes, productos, proveedores, '
        'pedidos, facturas, pagos y sucursales. '
        'Desarrollado con Django REST Framework.'
    ),
    'VERSION': '1.0.0',
    'SERVE_INCLUDE_SCHEMA': False,
    'COMPONENT_SPLIT_REQUEST': True,
    'TAGS': [
        {'name': 'Autenticación', 'description': 'Login, registro y JWT'},
        {'name': 'Clientes', 'description': 'CRUD de clientes'},
        {'name': 'Productos', 'description': 'CRUD de productos'},
        {'name': 'Proveedores', 'description': 'CRUD de proveedores'},
        {'name': 'Sucursales', 'description': 'CRUD de sucursales'},
        {'name': 'Pedidos', 'description': 'CRUD de pedidos'},
        {'name': 'Detalle Pedidos', 'description': 'CRUD de detalle de pedidos'},
        {'name': 'Facturas', 'description': 'CRUD de facturas'},
        {'name': 'Pagos', 'description': 'CRUD de pagos'},
    ],
}
```

2. JWT — Autenticación con tokens

Archivo	config/settings.py + api/autenticacion/views.py
Librería	django-rest-framework-simplejwt
Endpoints	POST /api/v1/auth/login/ POST /api/v1/auth/logout/
Descripción	El login devuelve un access token (60 min) y un refresh token (7 días). El logout añade el token a la blacklist. Todas las rutas protegidas exigen el header Authorization: Bearer <token>.

Captura de evidencia —

```
# — JWT —
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(
        minutes=env.int('JWT_ACCESS_TOKEN_LIFETIME_MINUTES', default=60)
    ),
    'REFRESH_TOKEN_LIFETIME': timedelta(
        days=env.int('JWT_REFRESH_TOKEN_LIFETIME_DAYS', default=7)
    ),
    'ROTATE_REFRESH_TOKENS': True,
    'BLACKLIST_AFTER_ROTATION': True,
    'AUTH_HEADER_TYPES': ('Bearer',),
}
```

3. Paginación

Archivo	config/pagination.py
Clase	StandardPagination (PageNumberPagination)
Parámetros	?page=2 ?page_size=5 (máx 100)
Descripción	Todas las respuestas de listado incluyen el objeto 'paginacion' con total de registros, página actual, total de páginas y links siguiente/anterior.

Captura de evidencia —

```
class StandardPagination(PageNumberPagination):
    page_size = 10
    page_size_query_param = 'page_size'
    max_page_size = 100

    def get_paginated_response(self, data):
        return Response({
            'success': True,
            'message': 'Consulta exitosa',
            'data': data,
            'paginacion': {
                'total': self.page.paginator.count,
                'pagina_actual': self.page.number,
                'total_paginas': self.page.paginator.num_pages,
                'siguiente': self.get_next_link(),
                'anterior': self.get_previous_link(),
                'por_pagina': self.get_page_size(self.request),
            }
        })
```

4. Filtros

Archivo	api/clientes/filters.py (mismo patrón en todos los módulos)
Librería	django-filter + DjangoFilterBackend
Ejemplo	GET /api/v1/clientes/?ciudad=Bogota&activo=true&tipo_documento=CC
Descripción	Cada módulo tiene su propio FilterSet. Los filtros de texto usan icontains (búsqueda parcial sin distinguir mayúsculas). Se permiten rangos de fechas con fecha_desde y fecha_hasta.

Captura de evidencia — request con filtros aplicados:

```
class ClienteFilter(django_filters.FilterSet):
    nombre = django_filters.CharFilter(lookup_expr='icontains')
    email = django_filters.CharFilter(lookup_expr='icontains')
    ciudad = django_filters.CharFilter(lookup_expr='icontains')
    tipo_documento = django_filters.ChoiceFilter(choices=Cliente.TIPO_DOCUMENTO)
    activo = django_filters.BooleanFilter()
    fecha_desde = django_filters.DateFilter(field_name='fecha_creacion', lookup_expr='gte')
    fecha_hasta = django_filters.DateFilter(field_name='fecha_creacion', lookup_expr='lte')

    class Meta:
        model = Cliente
        fields = ['nombre', 'email', 'ciudad', 'tipo_documento', 'activo']
```

5. Ordenamiento

Archivo	api/pedidos/views.py (mismo patrón en todos los ViewSets)
Backend	rest_framework.filters.OrderingFilter
Ejemplo	GET /api/v1/pedidos/?ordering=-fecha_pedido (descendente)
Descripción	Cada ViewSet declara ordering_fields con los campos permitidos y ordering con el orden por defecto. El guión (-) invierte el orden.

Captura de evidencia —

```
class PedidoViewSet(AuditoriaMixin, SoftDeleteMixin, RespuestaEstandarMixin, viewsets.ModelViewSet):
    serializer_class = PedidoSerializer
    filterset_class = PedidoFilter
    search_fields = ['numero_pedido', 'cliente_nombre', 'sucursal_nombre']
    ordering_fields = ['fecha_pedido', 'total', 'estado']
    ordering = ['-fecha_pedido']
    permission_classes = [AdminOVendedor]
```

6. Soft Delete — Eliminación lógica

Archivo	config/mixins.py + api/models_base.py
Campo	activo = BooleanField(default=True) en ModeloBase
Endpoint	DELETE /api/v1/clientes/{id}/ → activo=False (no borra el registro)
Descripción	El mixin SoftDeleteMixin sobrescribe destroy() para cambiar activo a False en lugar de ejecutar un DELETE SQL. El registro permanece en la base de datos para mantener la trazabilidad histórica.

Captura de evidencia —

```
class SoftDeleteMixin:
    """
    Sobreescribe destroy() para hacer eliminación lógica (activo=False).
    """

    def destroy(self, request, *args, **kwargs):
        instance = self.get_object()
        if not hasattr(instance, 'activo'):
            return respuesta_error('Este modelo no soporta eliminación lógica.')
        instance.activo = False
        instance.save(update_fields=['activo'])
        logger.info(
            f'SOFT-DELETE | {instance.__class__.__name__} id={instance.pk} | '
            f'usuario={request.user.username}'
        )
        return respuesta_eliminado()
```

7. Logging — Registro de peticiones

Archivo	config/middleware.py + config/settings.py (LOGGING)
Archivo log	logs/ventas_api.log
Descripción	LoggingMiddleware registra cada petición HTTP con método, ruta, código de respuesta, usuario autenticado, IP y duración en milisegundos. Los eventos CRUD también quedan registrados en el log.

Captura de evidencia —

```
class LoggingMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        inicio = time.time()

        usuario = (
            request.user.username
            if hasattr(request, 'user') and request.user.is_authenticated
            else 'Anónimo'
        )

        response = self.get_response(request)

        duracion_ms = round((time.time() - inicio) * 1000, 2)

        logger.info(
            f'{request.method} {request.path} | '
            f'Status: {response.status_code} | '
            f'Usuario: {usuario} | '
            f'IP: {self._get_ip(request)} | '
            f'Duración: {duracion_ms}ms'
        )
```


8. Exportación — Excel y CSV

Archivo	api/pedidos/views.py (mismo patrón en clientes y facturas)
Endpoint	GET /api/v1/pedidos/exportar/?formato=excel
Formatos	excel (.xlsx) csv (.csv)
Descripción	El action @exportar aplica los mismos filtros activos en el listado y genera el archivo para descarga. Se usa openpyxl para Excel y el módulo csv de Python para CSV.

Captura de evidencia —

```
@extend_schema(tags=['Pedidos'], summary='Exportar pedidos')
@action(detail=False, methods=['get'], url_path='exportar')
def exportar(self, request):
    fmt = request.query_params.get('formato', 'excel').lower()
    qs = self.filter_queryset(self.get_queryset())
    datos = PedidoSerializer(qs, many=True).data
    campos = ['id', 'numero_pedido', 'fecha_pedido', 'estado', 'total']
    if fmt == 'csv':
        resp = HttpResponse(content_type='text/csv')
        resp['Content-Disposition'] = 'attachment; filename="pedidos.csv"'
        w = csv.DictWriter(resp, fieldnames=campos)
        w.writeheader()
        for r in datos: w.writerow({c: r.get(c, '') for c in campos})
    return resp
```

9. Versionado de API

Archivo	config/urls.py + config/urls_v1.py
Prefijo	/api/v1/ en todas las rutas
Descripción	El versionado se implementa mediante prefijo de URL. Todos los endpoints de negocio están bajo /api/v1/. Una futura versión se agregaría como /api/v2/ sin romper la versión actual.

Captura de evidencia —

```
# API v1
path('api/v1/', include('config.urls_v1')),
```

10. Roles y Permisos

Archivo	config/permissions.py
Roles	Administrador Vendedor Bodega
Permisos	AdminOVendedor · EsAdministrador · EsBodega · SoloLecturaOAdmin
Descripción	Los permisos se asignan por grupo de Django. AdminOVendedor permite CRUD a Administradores y Vendedores. SoloLecturaOAdmin deja leer a cualquier autenticado pero solo Administrador puede escribir.

Captura de evidencia —

```
class EsAdministrador(BasePermission):
    """Solo usuarios del grupo Administrador."""
    def has_permission(self, request, view):
        return (
            request.user.is_authenticated and (
                request.user.is_superuser or
                request.user.groups.filter(name='Administrador').exists()
            )
        )
```

11. Nested Serializers

Archivo	api/pedidos/serializers.py + api/facturas/serializers.py
Descripción	En las respuestas GET, los campos cliente_detalle y sucursal_detalle devuelven el objeto completo anidado, no solo el ID. Para escritura (POST/PUT) se usan PrimaryKeyRelatedField. FacturaSerializer anida PedidoSerializer completo.

Captura de evidencia —

```
class PedidoSerializer(serializers.ModelSerializer):
    # Nested serializers (lectura)
    cliente_detalle = ClienteSerializer(source='cliente', read_only=True)
    sucursal_detalle = SucursalSerializer(source='sucursal', read_only=True)
    # Campos de escritura
    cliente = serializers.PrimaryKeyRelatedField(
        queryset=Pedido._meta.get_field('cliente').related_model.objects.filter(activo=True)
    )
    sucursal = serializers.PrimaryKeyRelatedField(
        queryset=Pedido._meta.get_field('sucursal').related_model.objects.filter(activo=True)
    )
```

12. Auditoría

Archivo	api/models_base.py + config/mixins.py (AuditoriaMixin)
Campos	creado_por (FK → User) modificado_por (FK → User) fecha_creacion fecha_modificacion
Descripción	ModeloBase es abstracto y lo heredan todos los modelos. AuditoriaMixin sobrescribe perform_create y perform_update para asignar automáticamente el usuario autenticado a creado_por y modificado_por.

Captura de evidencia —

```
class ModeloBase(models.Model):
    """
    Modelo abstracto que incluye los campos obligatorios de la guía:
    - activo
    - fecha_creacion
    - fecha_modificacion
    Además agrega auditoría de usuario (creado_por / modificado_por).
    """
    activo = models.BooleanField(default=True)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    fecha_modificacion = models.DateTimeField(auto_now=True)
    creado_por = models.ForeignKey(
        User, null=True, blank=True,
        on_delete=models.SET_NULL,
        related_name='%s_%s_creados' % (app_label, class_name)
    )
    modificado_por = models.ForeignKey(
        User, null=True, blank=True,
        on_delete=models.SET_NULL,
        related_name='%s_%s_modificados' % (app_label, class_name)
    )
```

13. Respuestas JSON Personalizadas

Archivo	config/responses.py + config/exceptions.py + config/mixins.py
Estructura	{ success: bool, message: string, data: object null }
Descripción	Todas las respuestas de la API siguen el mismo formato JSON. Las funciones respuesta_exitosa, respuesta_creada, respuesta_error y respuesta_eliminado son usadas por RespuestaEstandarMixin y por el manejador de excepciones personalizado.

Captura de evidencia —

```
def respuesta_exitosa(data=None, mensaje='Operación exitosa', codigo=status.HTTP_200_OK):
    return Response({
        'success': True,
        'message': mensaje,
        'data': data,
    }, status=codigo)

def respuesta_creada(data=None, mensaje='Registro creado exitosamente'):
    return Response({
        'success': True,
        'message': mensaje,
        'data': data,
    }, status=status.HTTP_201_CREATED)

def respuesta_error(mensaje='Error en la operación', errores=None, codigo=status.HTTP_400_BAD_REQUEST):
    return Response({
        'success': False,
        'message': mensaje,
        'errors': errores,
    }, status=codigo)

def respuesta_no_encontrado(mensaje='Registro no encontrado'):
    return Response({
        'success': False,
        'message': mensaje,
        'data': None,
    }, status=status.HTTP_404_NOT_FOUND)
```